VLSI Technology 

Q3TUI:

Product Description

System Architecture and Main Flows

Version 1.0 | November 6, 2026

Contents

CONTENTS 2

TABLE OF TABLES 5

TABLE OF FIGURES 6

REVISION HISTORY 7

PREFACE 8

1. PRODUCT OVERVIEW 9

1.1. What Is Q3TUI 9

1.2. Problem and Value 9

1.3. Inputs and Outputs 9

1.4. Current Scope 10

2. SYSTEM ARCHITECTURE 11

2.1. Block Diagram 11

2.2. Source Code Structure 12

2.3. Data Flow of an LLM Stage 13

2.4. LLM Providers and Gateway 14

2.5. Role-Based EDA Tools 15

3. PIPELINE ENGINE AND FLOWS 16

3.1. Steps, Stages, and Gates 16

3.2. Flows as Data 16

3.3. Sessions 17

4. MAIN FLOWS 18

4.1. Flow Chain and Unattended Runs 18

4.2. Spec Flow: From Intent to Locked Configuration 19

4.3. DV Flow: RTL-Independent Testbench 21

4.4. Sanity Flow: CDC/RDC and Waiver Review 22

4.5. FPGA and ASIC Flows: PPA and Synthesis 24

4.6. Running Multiple Flows in Parallel 25

5. CORE MECHANISMS 26

5.1. Changes Propagate as Updates, Not Reruns 26

5.2. Gates, Questions, and Decisions 27

5.3. Fix Loop 29

5.4. Deterministic First 29

5.5. RTL and Testbench Independence 30

5.6. Team Mode 31

5.7. Project History 31

6. PROJECT DIRECTORY STRUCTURE 32

7. CONFIGURATION AND USAGE 33

7.1. Installation 33

7.2. Configuration Layers 33

7.3. Quick Start 34

7.4. Main CLI Commands 34

7.5. Development 34

8. CURRENT LIMITATIONS AND ROADMAP 36

APPENDIX A. ACRONYMS AND TERMS 37

Table of Tables

Table 1-1: Q3TUI Core Values1-1 9
Table 1-2: Outputs of the Main Flows1-2 10
Table 2-1: Architecture Layers and Source Locations2-1 12
Table 2-2: Main Directories in the q3tui Package2-2 12
Table 2-3: LLM Stage Contract (q3tui.llm.runtime)2-3 13
Table 2-4: llm.provider Values2-4 14
Table 2-5: Tool Roles Used by Flows (Examples)2-5 15
Table 3-1: Gate Modes3-1 16
Table 3-2: Built-In Flows3-2 16
Table 4-1: Steps in the Spec Flow4-1 19
Table 4-2: Steps in the RTL Flow4-2 20
Table 4-3: The Two DV Testbench Types (tb Step testbench Option)4-3 21
Table 4-4: Steps in the Sanity Flow4-4 23
Table 4-5: Steps in the FPGA and ASIC Flows4-5 24
Table 5-1: Question Types and Owning Steps5-1 28
Table 5-2: What Each Step Can See (Spec and RTL Flows)5-2 30
Table 5-3: Team Mode Roles (llm/roles.py)5-3 31
Table 6-1: Main Directories and Owners6-1 32
Table 7-1: Configuration Precedence (Later Layers Win)7-1 33
Table 7-2: Main q3tui Commands7-2 34
Table 8-1: Current Limitations8-1 36
Table 8-2: Acronyms and Terms8-2 37

Table of Figures

Figure 2-1: Q3TUI Architecture Block Diagram2-1 11
Figure 2-2: Data Flow for an LLM Stage and Its Tools2-2 13
Figure 2-3: LLM Providers and LiteLLM Gateway2-3 14
Figure 4-1: Main Flow Chain and Reruns After RTL Changes4-1 18
Figure 4-2: Steps in the Spec and RTL Flows4-2 19
Figure 4-3: Steps in the DV Flow and Failed-Test Triage4-3 21
Figure 4-4: Steps in the Sanity Flow and Fix Loop4-4 22
Figure 4-5: Steps in the FPGA and ASIC Flows4-5 24
Figure 5-1: A Step Updates Instead of Rerunning5-1 26
Figure 5-2: Gates, Gate Modes, and Question Routing5-2 28
Figure 5-3: Fix Loop Between a Checking Step and Its Owner5-3 29
Figure 5-4: RTL and Testbench Independence5-4 30

Revision History

Table 0-1: Revision History

Version	Date	Revision details
1.0	November 6, 2026	Initial release: description of Q3TUI 0.1.0 (Beta), its architecture, and main flows, based on repository commit 60408e4.

Preface

This document describes Q3TUI, VLSI Technology's multi-agent chip design and verification platform. It explains the problem the product solves, how it is organized, and how its main flows move from an idea (intent) or an existing specification to checked RTL, a testbench, CDC/RDC results, and PPA estimates.

The content was checked against the repository source and design documentation (CLAUDE.md, README.md, docs/spec/, and the q3tui package). Where repository documentation and source code disagree, this document follows the source and notes the discrepancy in Chapter 8. Directory names, filenames, commands, and configuration parameters remain in English.

The figures are generated by docs/images/product/gen_figures.py in the repository. Edit the script and run it again to update them.

1. Product Overview

1.1. What Is Q3TUI

Q3TUI (Q3 = Quality · Quick · Quorum) is a terminal user interface (TUI) with a CLI that takes a hardware IP block from intent (a few paragraphs describing the idea) or an existing specification to checked synthesizable RTL, a testbench, CDC/RDC results, and PPA estimates. Work is divided into flows; each flow is a pipeline of steps with gates where users pause to make decisions.

All LLM calls run through the Claude Agent SDK. EDA tools (lint, synthesis, simulation, CDC, and others) sit behind vendor-agnostic role adapters; Q3TUI ships with no EDA tools. The product is from VLSI Technology, licensed under Apache 2.0, and is in Beta (version 0.1.0).

1.2. Problem and Value

Using an LLM to write RTL often runs into three problems: results are hard to control (a "black box"), every requirement change triggers a rerun from the beginning and discards approved work, and verification is not independent of design. Q3TUI addresses these with five principles.

Table 1-1: Q3TUI Core Values1-1

Principle	Meaning in the product
Human review gates	A step that needs a user decision pauses at a gate. Modes are human, auto, auto_answer, or none, configured per project or run.
Deterministic first	Parsing, rule checks, lint, formatting, simulation, and synthesis are code and tools. The LLM receives their results as ground truth and never has the final say on compliance.
Update, do not rerun	Changing one requirement updates only affected parts; approved content remains unchanged.
Independent verification	RTL cannot see the testbench; the testbench cannot see RTL until final integration in the UVM backend. Hooks enforce this, not prompts alone.
One engine, three interfaces	The TUI, CLI, and in-TUI chat assistant all call the same operation layer (core/ops.py).

1.3. Inputs and Outputs

Users provide a short intent or a specification; each flow continues from the artifacts of the previous flow. Every step reads regular project files, so work can begin at any point:

with an existing specification, start at parse; q3tui run executes only missing or stale work, much like make.

Table 1-2: Outputs of the Main Flows1-2

Flow	Output
`spec (default)`	Approved specification, numbered REQ-NNN requirements, parameters, and module map.
`rtl`	RTL checked against user rules, formatted, style-linted, and smoke-tested; registers generated by APB-CSR-Generator.
`dv`	Spec-driven testbench (plain SystemVerilog or UVM using an existing VIP), run in the user's simulator.
`sanity`	CDC/RDC analysis; each violation is classified as a waiver, RTL fix, or question.
`fpga, asic`	PPA estimate for an FPGA part; standard-cell synthesis netlist with LEC.

1.4. Current Scope

- New IP blocks designed from scratch, with up to a few clock domains; the CLI and Textual TUI share one pipeline engine.
- Users configure role-based EDA tools (lint, synth, sim, run, cdc, rdc, and others) in config/tools.json; q3tui tools init writes a reference configuration to get started.
- According to docs/spec/README.md, future directions include formal verification, upgrading existing RTL, and a web UI.

2. System Architecture

2.1. Block Diagram

Figure 2-1 shows Q3TUI's layers. All user actions pass through core/ops.py; the engine runs the flow steps; and each step uses three kinds of resources: the LLM runtime, deterministic HDL processing, and role-based EDA tools.

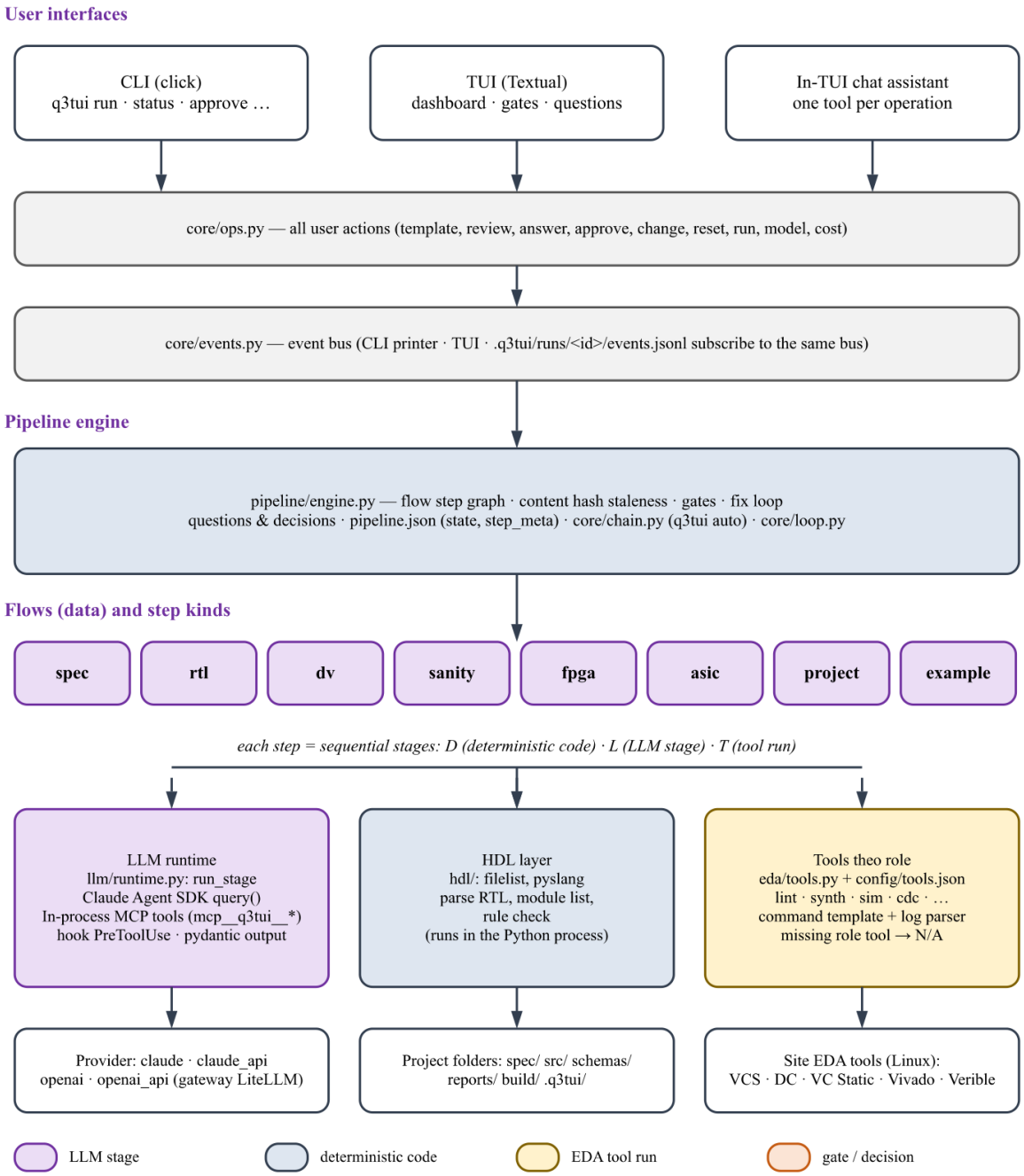


Figure 2-1: Q3TUI Architecture Block Diagram2-1

Table 2-1: Architecture Layers and Source Locations2-1

Layer	Role	Source
User interface	CLI (click), TUI (Textual), and chat assistant with one tool per operation; all call core/ops.py.	`core/cli.py, tui/, llm/assistant.py`
Event bus	Typed events; the CLI printer, TUI, and events.jsonl all subscribe.	`core/events.py`
Pipeline engine	Flow step graph, content-hash staleness, gates, fix loop, questions, and pipeline.json state.	`pipeline/engine.py, pipeline/state.py`
Flow and step kinds	Flows are data (YAML files, directories, or Markdown); each step uses a registered kind.	`flows/, q3tui/flows/<name>/, steps/`
LLM runtime	One run_stage function for every LLM call; selects provider, model, and session; file permissions are enforced by hooks.	`llm/runtime.py, llm/provider.py, llm/roles.py`
HDL layer	File lists and RTL parsing with pyslang; module lists and rule checks.	`hdl/`
Role-based tools	Roles map to command templates and log parsers; diagnostics are normalized.	`eda/tools.py, eda/presets/`
Long-running orchestration	`q3tui auto runs the flow chain; q3tui loop runs the sanity fix loop.`	`core/chain.py, core/loop.py`

2.2. Source Code Structure

The q3tui/ package is at the repository root (Python ≥ 3.12, managed with uv). Table 2-2 lists its main directories.

Table 2-2: Main Directories in the q3tui Package2-2

Directory	Contents
`core/`	CLI, config, settings, project layout, events, stats, language, ops, history, importer, chain, loop, and keys; compat.py handles Linux/Windows differences at runtime (file locks, process groups, pid_alive, and UTF-8 mode).
`pipeline/`	Engine (step graph, staleness, gates), state.py (pipeline.json), and base.py (StepDef class).
`llm/`	Runtime, roles, prompts, providers and gateway, model list, chat assistant, and internal design tools.
`eda/`	Role-based tools (tools.py), legacy adapters (slang, Synopsys), VCD parser, and tools.json presets.
`hdl/`	`filelist.py, parse.py (pyslang), and loops.py.`
`flows/`	One directory per flow: spec, rtl, dv, sanity, fpga, asic, project, and example; each step has step.py, md/, rules/, schemas/, and scripts/.
`steps/`	Spec step and shared components (common.py: questions, answers, and scoped change requests).
`tui/`	Textual app, panels registered by step kind, pickers, dialogs, and background runner.

2.3. Data Flow of an LLM Stage

A step consists of sequential stages: D (deterministic code), L (an LLM stage with a limited Agent SDK tool set and structured output), and T (an EDA tool run). For stage L, the step declares Stage(...); run_stage handles the rest (Figure 2-2).

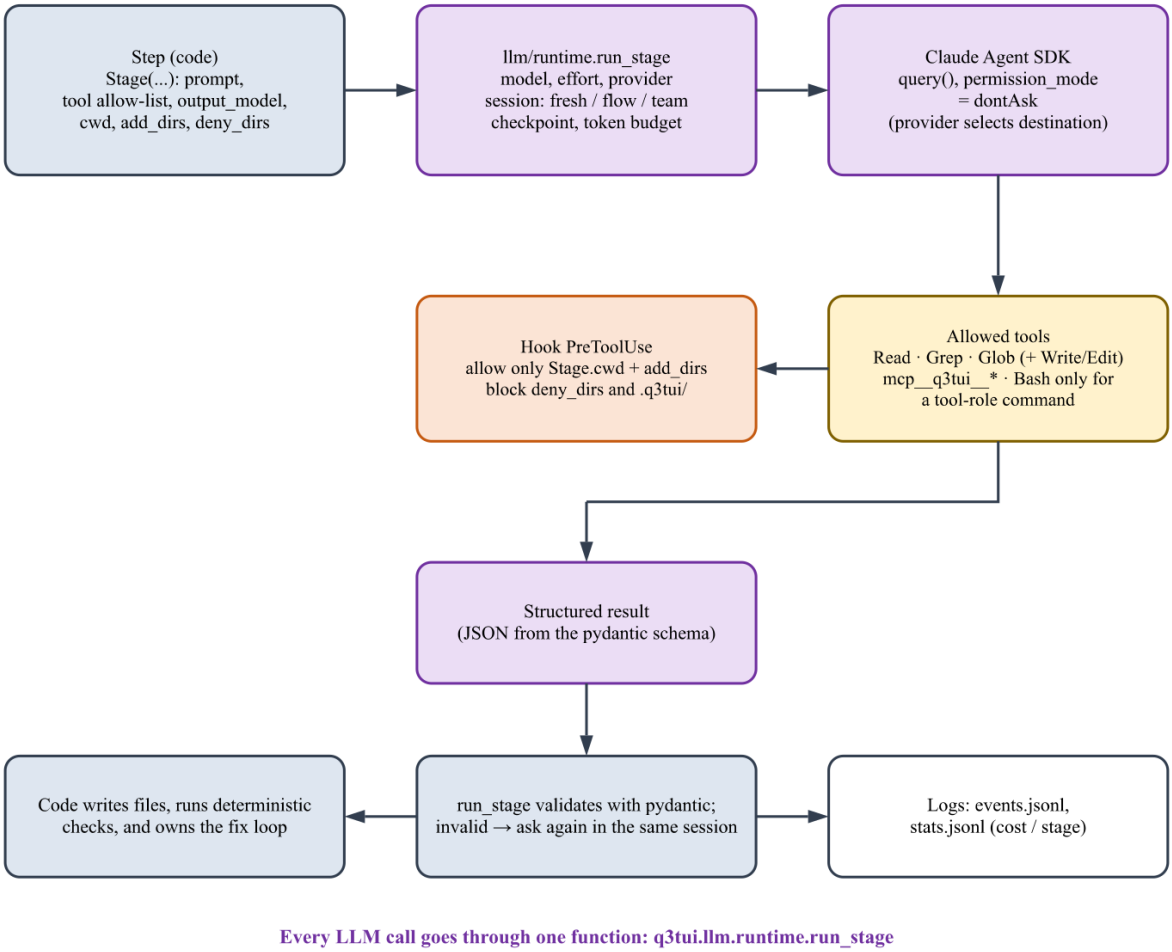


Figure 2-2: Data Flow for an LLM Stage and Its Tools2-2

Table 2-3: LLM Stage Contract (q3tui.llm.runtime)2-3

Aspect	Contract
Harness	`claude_agent_sdk.query()` for each stage; fresh session (one per stage), flow (one per flow), or team mode.
Model	`llm.model` (claude-sonnet-5-5 by default) and `llm.effort`; per-step overrides through `llm.steps`.
Tools	Explicit allow-list. Read-only stages: Read, Grep, Glob; code-writing stages also get Write/Edit; Bash is limited to one tool-role command.
File access	PreToolUse hook confines access to Stage.cwd + add_dirs, blocks deny_dirs, and always blocks .q3tui/. Hooks enforce access; prompts are a fallback.

Aspect	Contract
Q3TUI tools	In-process MCP server (mcp__q3tui__*): design information, safe glob, and step-specific tools.
Permissions	`permission_mode="dontAsk": only allow-listed actions can run.`
Output	Pydantic model → JSON schema → output_format; results are validated and invalid output is requested again in the same session.
Budget and logs	`Optional max_budget_usd; llm.stage_token_budget flags runaway stages; all messages go to .q3tui/runs/<id>/events.jsonl.`

2.4. LLM Providers and Gateway

Every stage runs through the Claude Agent SDK; llm.provider determines where each request goes (Figure 2-3). The two OpenAI providers use a local LiteLLM proxy (gateway) that Q3TUI starts; LiteLLM runs in its own venv at ~/.q3tui/gateway/venv.

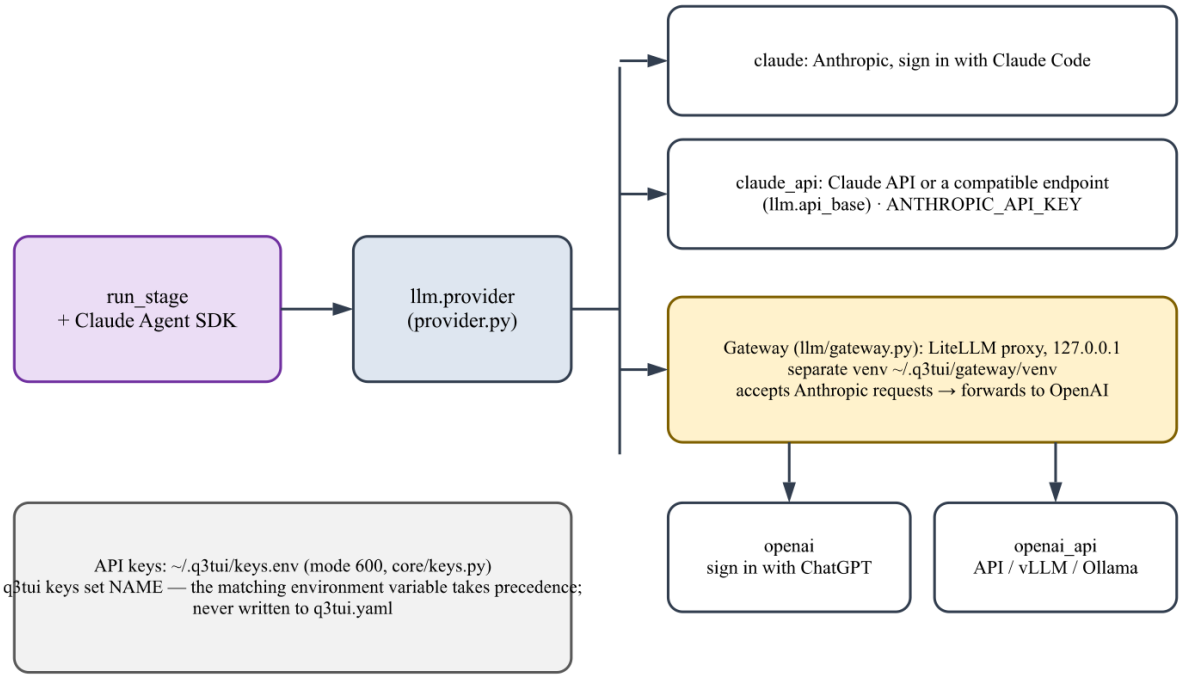


Figure 2-3: LLM Providers and LiteLLM Gateway2-3

Table 2-4: llm.provider Values2-4

`llm.provider`	Request is routed to	Required
`claude (default)`	Anthropic via Claude Code login (Claude subscription)	Claude Code login
`claude_api`	Claude API or compatible endpoint (llm.api_base)	`ANTHROPIC_API_KEY`
`openai`	OpenAI via ChatGPT login (Codex models), through the gateway	`q3tui gateway login`

llm.provider	Request is routed to	Required
openai_api	OpenAI API or compatible endpoint (vLLM, Ollama, company endpoint), through the gateway	OPENAI_API_KEY

API keys are stored in ~/.q3tui/keys.env (mode 600), set with q3tui keys set NAME or the Settings dialog. A matching environment variable takes precedence, and keys are never written to q3tui.yaml.

2.5. Role-Based EDA Tools

A step calls a role (run_role(tools, "lint", …)); it never calls a vendor binary directly. The project's config/tools.json is the single source of configuration: each role has a command template and a log parser; diagnostics are normalized to {severity, code, file, line, message}, and raw logs are retained. An unconfigured role or missing capability (supports: [sva]) is reported as N/A, never as a pass.

Table 2-5: Tool Roles Used by Flows (Examples)2-5

Role	Used in	Notes
lint	rtl flow	RTL lint; requires zero warnings.
sim, run	rtl (smoke), dv (plain)	Compile and run each test case (+TC=<id>).
sim_uvm, run_uvm, sw_build	dv (uvm, prepare)	UVM testbench; test programs/data (RISC-V GCC + Spike).
cdc, rdc	sanity	VC Static (vc_static_shell -f).
asic_synth, asic_lec	asic	Design Compiler (dc_shell -f), Formality (fm_shell -f).
Vivado / Yosys + nextpnr / Quartus	fpga	Selected by FPGA part; out-of-context synthesis and timing/utilization reports.

Tool logs stream to files instead of being held in memory: duplicate lines are retained up to 20 times and then counted; after 32 MB, only the last 4 MB are kept. A timeout (tools.timeout_s) terminates the tool's entire process tree.

3. Pipeline Engine and Flows

3.1. Steps, Stages, and Gates

The engine (pipeline/engine.py) runs a flow's steps in dependency order. It does not hardcode step names: it queries the flow through engine.pipeline_steps, engine.label, and engine.gate_modes(). A step is a StepDef subclass that declares inputs and outputs; the engine decides when it runs.

Table 3-1: Gate Modes3-1

Mode	Behavior
`human`	Pause for your review: approve, answer a question, edit, request a change, or defer.
`auto`	Auto-approve completed steps; open blocking questions still pause the run (non-blocking questions use defaults).
`auto_answer`	Unattended: approve everything and answer every question with its default (suggestions are accepted; pipeline.max_fix_iterations bounds the run).
`none`	No review gate (blocking questions still stop the pipeline).

The flow file sets the default mode; pipeline.gate_modes: {step: mode} overrides it. Change a mode with q3tui gate STEP MODE, the TUI's g key, or Settings → Gates; each route calls Ops.set_gate_mode.

3.2. Flows as Data

pipeline.flow: <name|path> selects a flow file from the project's flows/ directory, ~/.q3tui/flows/, or the built-ins. A flow file lists step kinds (steps.KINDS, register_kind), dependencies, gate modes, options, gap_targets (which step owns a question), and sees (which decisions each step can see). A flow can be a directory, YAML file, or Markdown file; users can define flows from registered step kinds.

Table 3-2: Built-In Flows3-2

Flow	Steps	Notes
`spec (default)`	spec → parse → config	Gate 1 at parse (human); Gate 2 at config (auto when all constraints pass).
`rtl`	spec → rules → parse → config → csr → rtl → format → style_lint → smoke	Reuses the three steps from the spec flow.
`dv`	spec → parse → config → testplan → tb → prepare → simulate	Two testbench types: plain and uvm.
`sanity`	sdc → setup → (cdc, rdc) → waivers → check	Independent CDC and RDC lanes.
`fpga`	fpga_port → sdc → fpga_synth → timing_util_report → ppa_optimize	PPA estimate; does not deploy to a board.

Flow	Steps	Notes
`asic`	asic_port → sdc → asic_synth → asic_lec	Design Compiler and Formality.
`project`	project, tools, fpga, libraries	Project setup: tool availability and ASIC libraries.
`example`	spec → parse	Minimal folder flow to copy.

3.3. Sessions

session: fresh (the default) gives each LLM stage its own session; session: flow uses one persistent Claude session for the whole flow (role flow). Reviewers (stages named *review*) and steps listed in fresh_steps always run fresh. In particular, session: flow removes isolation for testbench smoke because that session has already seen RTL, so no built-in flow uses it.

4. Main Flows

4.1. Flow Chain and Unattended Runs

The default pipeline.chain is spec → rtl → dv → sanity → fpga (Figure 4-1). Each flow continues from the previous flow's artifacts, and every flow can also run on its own (q3tui --flow rtl run).

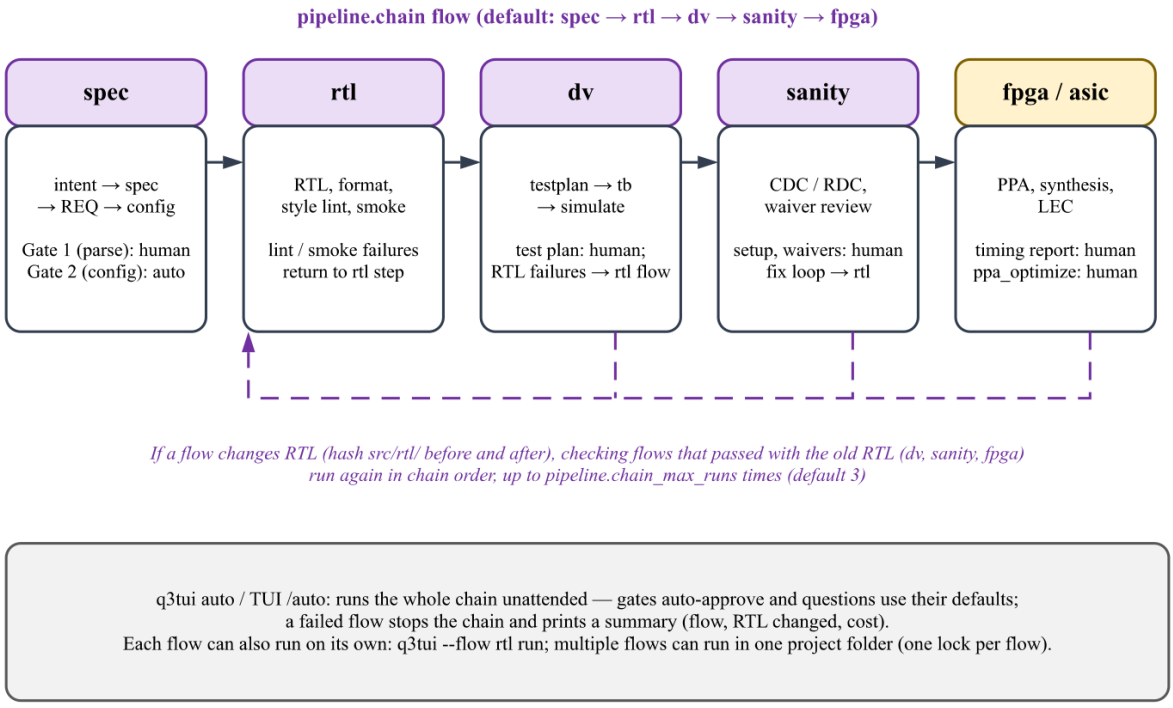


Figure 4-1: Main Flow Chain and Reruns After RTL Changes4-1

q3tui auto (TUI: /auto) runs the full chain unattended: gates auto-approve and questions receive their default values. It finishes only when every flow has passed against the final RTL:

- A flow passes when its run finishes with every step done. Sanity runs its fix loop (q3tui loop) and passes when no open violations remain.
- A flow that changes RTL (src/rtl/ is hashed before and after each flow) causes checking flows (dv, sanity, fpga) that passed with the old RTL to rerun in chain order. spec and rtl run once because their fixes are handled by rtl.
- Each flow runs at most pipeline.chain_max_runs times (3 by default). If a flow does not pass, the chain stops and prints a table showing the flow, whether RTL changed, and the cost.

- Tool failures (crash, timeout, missing tool: failure.kind == "tool") go directly to the LLM tool_fix stage with full context. That stage may change only tool configuration or options, never RTL or the spec; it gets at most two attempts per flow run.

4.2. Spec Flow: From Intent to Locked Configuration

The spec flow turns an intent (or your specification) into REQ-NNN requirements and a locked configuration. The RTL flow reuses these three steps, then adds RTL generation and checking steps (Figure 4-2).

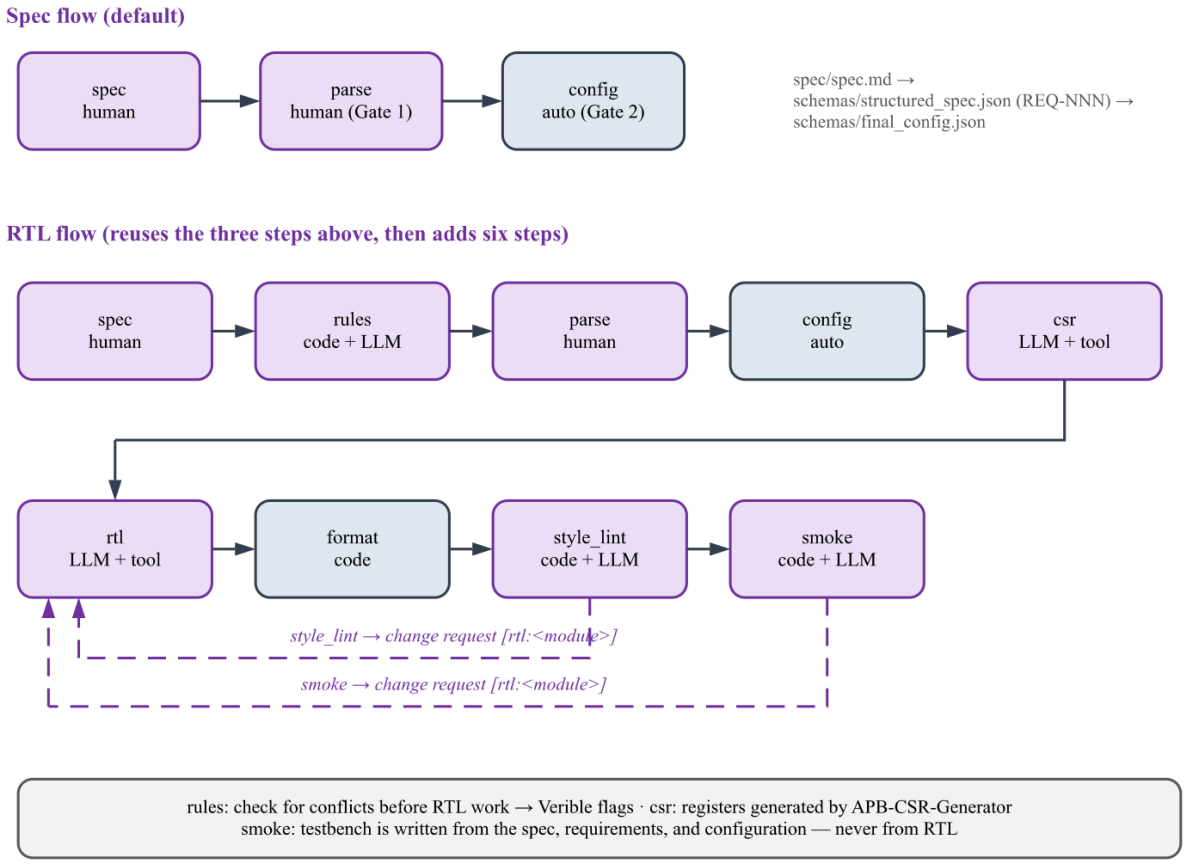


Figure 4-2: Steps in the Spec and RTL Flows4-2

Table 4-1: Steps in the Spec Flow4-1

Step	Kind	Purpose	Gate
`spec`	`spec`	Writes spec/spec.md from intent or supplied documents, using the sectioned template.	human

Step	Kind	Purpose	Gate
parse	vlsit_parse	An LLM stage extracts requirements (REQ-NNN, category, ambiguity_score), parameters, module mapping, and ambiguity questions; code assembles and validates schemas/structured_spec.json. Requirements with ambiguity scores > 0.3 need review.	human (Gate 1)
config	vlsit_config	Locks parameters (defaults or specification overrides); code evaluates constraints C1–C5 and writes schemas/final_config.json.	auto (Gate 2)

Table 4-2: Steps in the RTL Flow4-2

Step	Purpose
rules	Read the layered rules, check for conflicts before RTL generation, and write Verible flags (verible.flags, format.flags).
csr	Software-visible registers: draft a register map from the spec, validate it, generate it with APB-CSR-Generator, and pass it to rtl.
rtl	Generate each module in src/rtl/; check rules in code, run lint (role lint) and synthesis; update changed units instead of regenerating.
format	Run verible-verilog-format with format.flags; code only.
style_lint	Run Verible lint with verible.flags and an LLM review; findings go to the owning step as scoped change requests.
smoke	Write a basic testbench from the spec only, then compile it with RTL and run it; findings return to rtl the same way.

The rtl → format → style_lint → smoke sequence runs automatically. When style_lint or smoke finds an issue, it sends a scoped change request [rtl:<module>] … to rtl to update only that module; format, style_lint, and smoke then rerun. options.max_rounds (pipeline.max_fix_iterations by default) bounds this loop. If the limit is reached, the step fails; later steps are never allowed to pass by default.

4.3. DV Flow: RTL-Independent Testbench

The DV flow builds a testbench from the spec, requirements, and configuration, never from RTL (Figure 4-3). RTL remains a black-box DUT until the testbench is compiled with it.

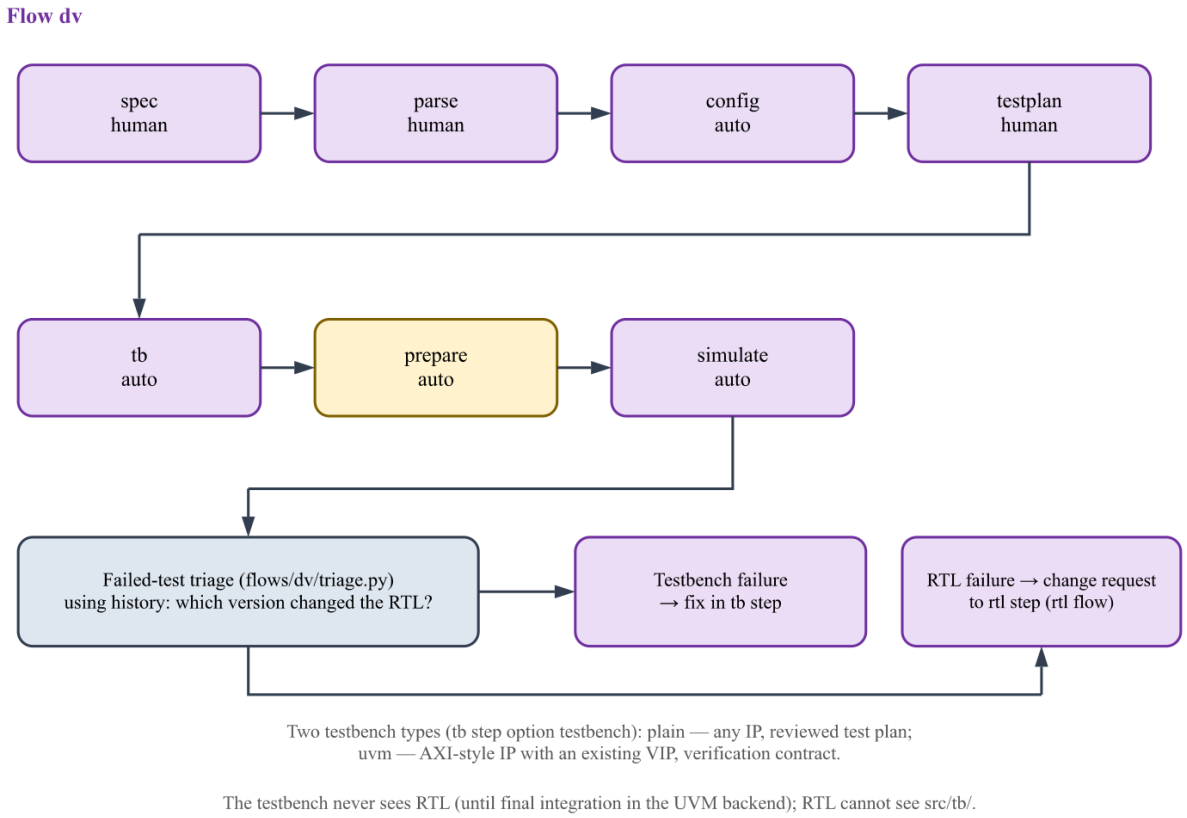


Figure 4-3: Steps in the DV Flow and Failed-Test Triage4-3

Table 4-3: The Two DV Testbench Types (tb Step testbench Option)4-3

	plain (any IP, no VIP required)	**uvm (existing protocol VIP)**
Auto-selected when	No VIP location is set	A VIP location is set
testplan	**Test plan: test cases linked to the REQs they check; uncovered requirements become blocking questions.**	**Verification contract: tests, goals, scoreboard rules, coverage goals, and open questions, frozen by hash.**
tb	tb_top and one file per test case in src/tb/, compiled with RTL using role sim.	Architecture, VIP integration, and scaffold; RTL is then the DUT for conformance, wrapper, and compilation (sim_uvm).
simulate	Run each test case separately (+TC=<id>); status comes from TC_RESULT lines.	Smoke and directed tests (run_uvm); status comes from the UVM report.

	plain (any IP, no VIP required)	uvm (existing protocol VIP)
Scope	Any IP.	AXI-style memory-mapped IP (ID / ADDR / DATA width parameters); other IPs are rejected.

The prepare step builds the programs and data that plain testbench tests load (src/sw/, built by sw_build using RISC-V GCC and Spike). When a test fails, triage (flows/dv/triage.py) assigns ownership and checks project history to see which RTL versions changed since that test last passed. RTL failures become change requests to the rtl step of the RTL flow; unattended runs execute the fix loop within the same run. Example: examples/axi4_sram_ctrl.

4.4. Sanity Flow: CDC/RDC and Waiver Review

The sanity flow reads RTL (src/rtl/, read-only) and shares SDC with the FPGA/ASIC flows (constraints/sdc/). It runs static CDC and RDC analysis with Synopsys VC Static, lists every violation in a table, and classifies each as a waiver, RTL fix, spec question, SDC request, or user question (Figure 4-4).

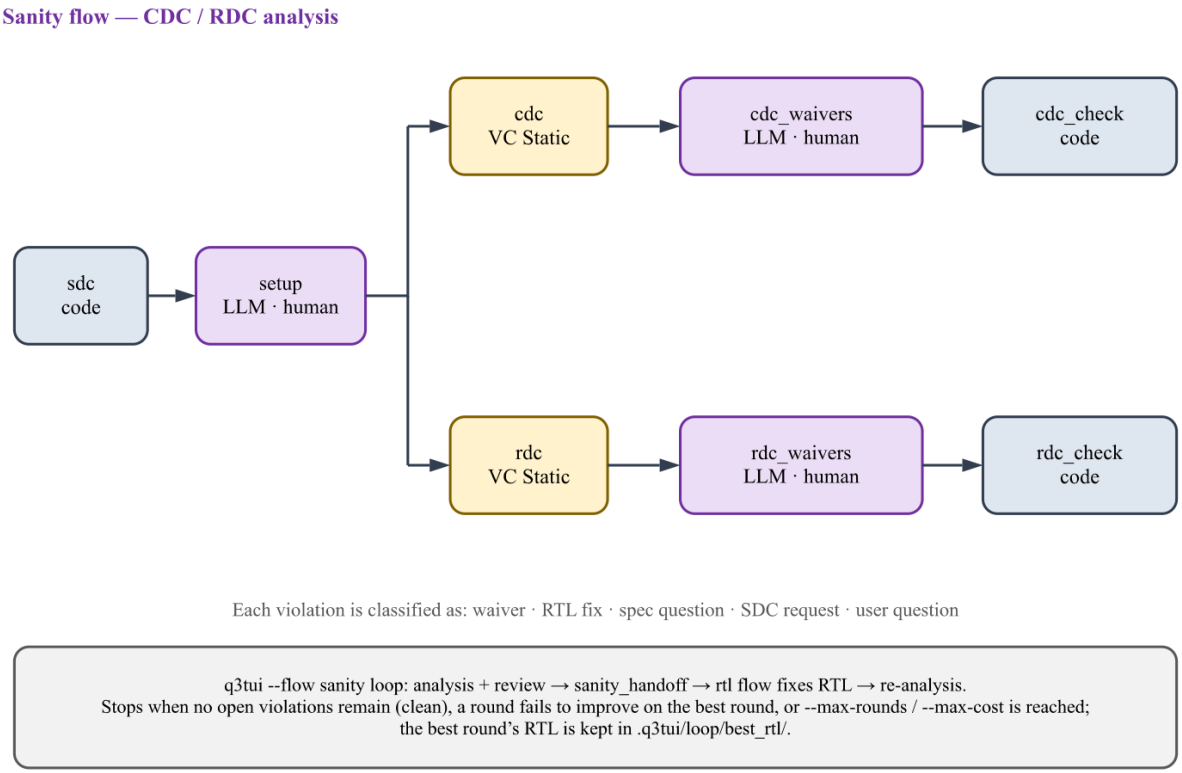


Figure 4-4: Steps in the Sanity Flow and Fix Loop4-4

Table 4-4: Steps in the Sanity Flow4-4

Step	Purpose
`sdc`	SDC shared by both lanes; render from sdc.json, or use your sdc.tcl unchanged when sdc.json is absent.
`setup`	A read-only LLM stage decides the analysis environment (resets, black boxes, quasi-static signals, constants, reset synchronizers); code checks names and runs a tool probe; you review at the gate.
`cdc / rdc`	The role tool runs a Q3TUI-generated script; each fatal, error, or warning becomes a violation; write reports/<role>_report.md/.csv.
`cdc_waivers / rdc_waivers`	A read-only LLM stage classifies each violation; you review at the gate.
`cdc_check / rdc_check`	Code only: rerun the tool with waivers loaded; fail unless open violations exactly match those not waived.

q3tui --flow sanity loop repeats analysis and review → sanity_handoff → RTL fixes → re-analysis. It stops when no open violations remain (clean), a round does not improve on the best round (no improvement), or --max-rounds or --max-cost is reached. The best round's RTL is kept in .q3tui/loop/best_rtl/ and can be restored with --restore-best.

4.5. FPGA and ASIC Flows: PPA and Synthesis

These flows take RTL from the RTL flow and share constraints/sdc/. They return detected issues as change requests (Figure 4-5).

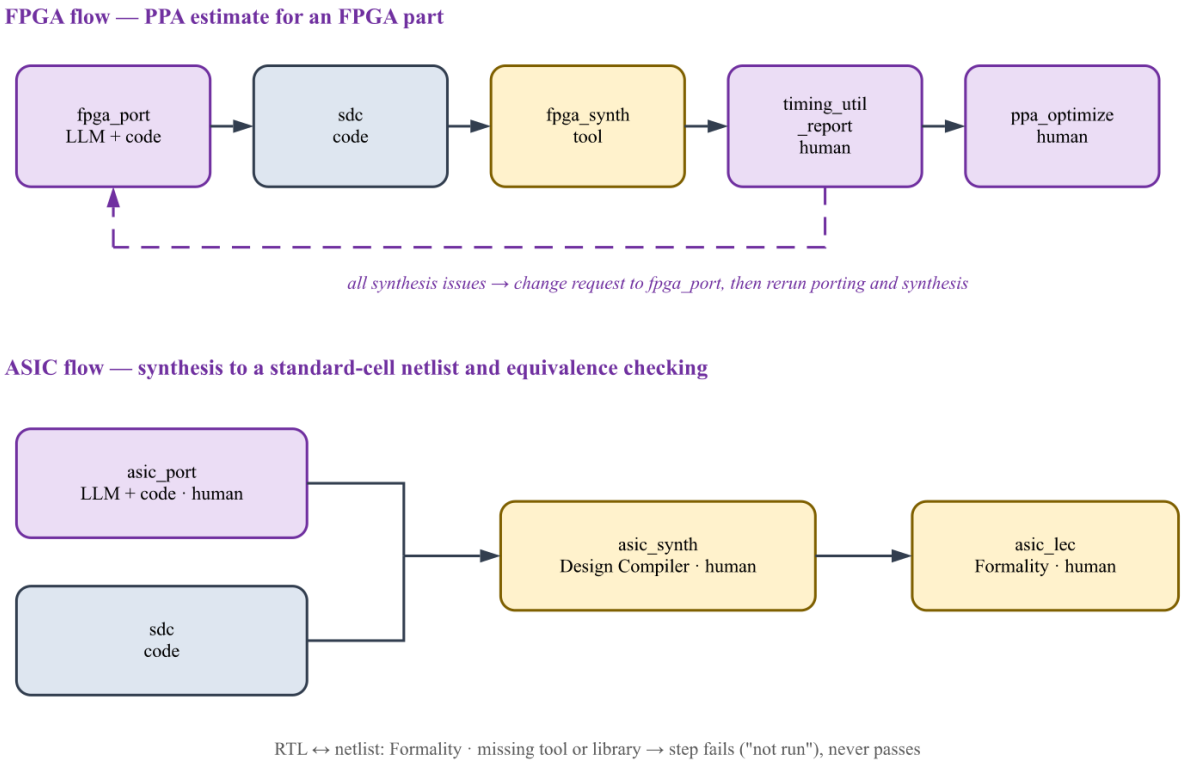


Figure 4-5: Steps in the FPGA and ASIC Flows4-5

Table 4-5: Steps in the FPGA and ASIC Flows4-5

Flow	Step	Purpose
fpga	`fpga_port`	Scan for constructs unsupported by the selected part's tools (memory initialization files, another vendor's primitives), then use the LLM and tools to port them; synthesis issues return here.
fpga	`sdc`	`Convert constraints/sdc/sdc.json to Tcl files (code only).`
fpga	`fpga_synth`	Family-specific script: Vivado out-of-context, Yosys + nextpnr, or Quartus Prime; writes schemas/fpga_synth.json.
fpga	`timing_util_report`	Read reports for LUT/FF/BRAM/DSP, WNS/Fmax, and critical paths; compare with targets; human gate.
fpga	`ppa_optimize`	When targets are missed, a read-only LLM stage proposes fixes; human gate.
asic	`asic_port`	Scan src/rtl/ for unsupported ASIC constructs (SRAM wrappers, synchronizer cells, ICG, and others); the porter creates an RTL work order; human gate.
asic	`asic_synth`	Design Compiler: write netlist build/syn/asic/netlist/<top>.{v,ddc,sdc,svf} and area/timing/power reports; human gate.
asic	`asic_lec`	Formality proves the netlist equivalent to RTL; human gate.

If a tool or library is missing, the step fails with status "not run"; it is never counted as passed.

4.6. Running Multiple Flows in Parallel

DV, FPGA, and sanity can run concurrently in one project folder, each as a separate process. This is safe because each flow has its own run lock (.q3tui/run.<flow>.lock); pipeline.json is saved under a file lock and merged step by step; a step runs in only one process at a time; and one run cannot have its design changed by another run's step (steps in pipeline.design_flows, [rtl] by default, wait for an exclusive turn before editing the design).

5. Core Mechanisms

5.1. Changes Propagate as Updates, Not Reruns

The rule for every step is that downstream artifacts are updated when inputs change: only affected parts are rewritten, while everything else remains byte-for-byte unchanged (Figure 5-1). A step is rebuilt from scratch only on its first run, after reset, or with --regenerate <step>.

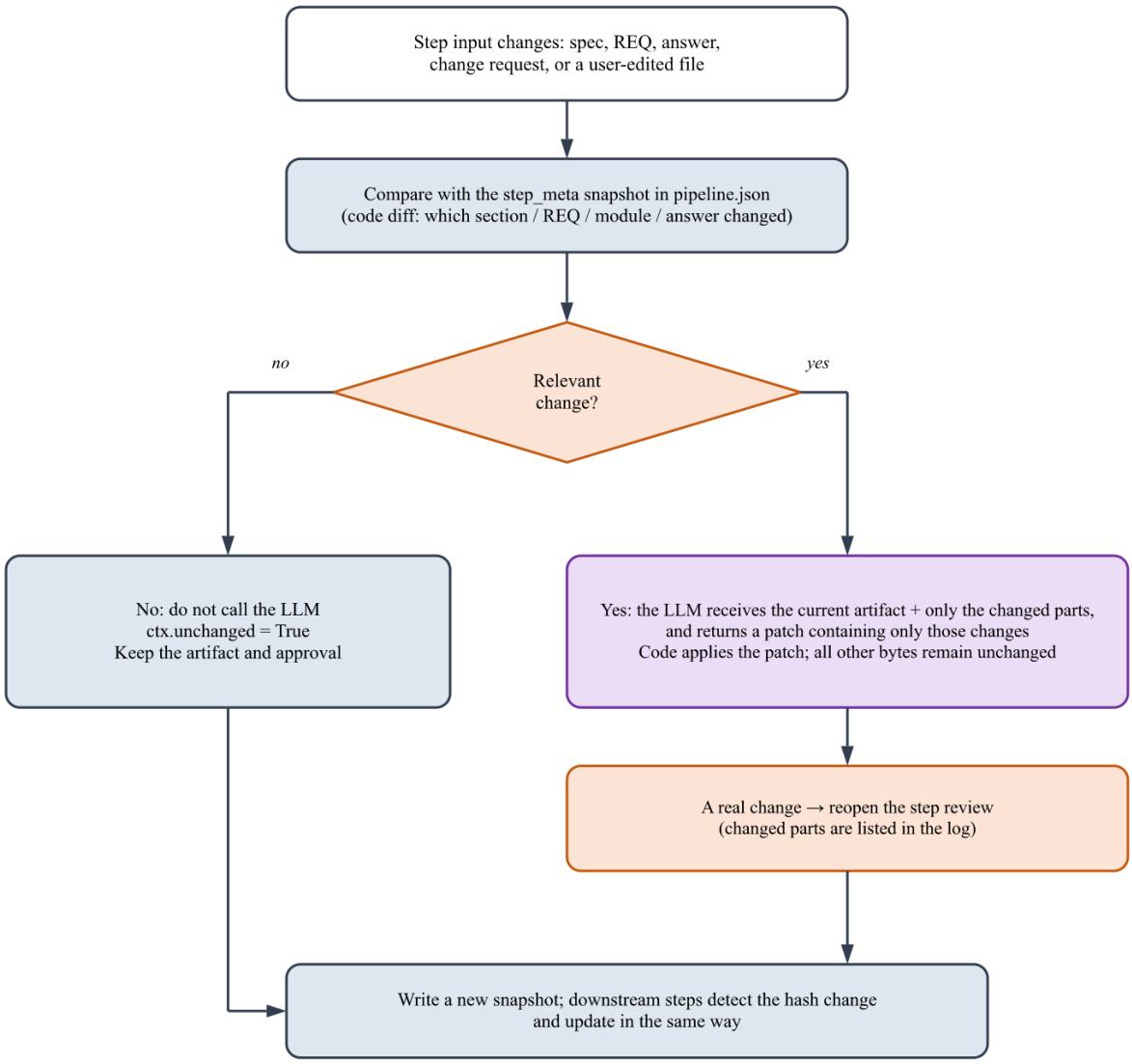


Figure 5-1: A Step Updates Instead of Rerunning5-1

- **Snapshot:** after each run, the step records what it was built from (hashes of spec sections, REQ, parameters, and answers) in pipeline.json (step_meta).

- **Code-based diff:** the next run compares inputs with the snapshot. If nothing relevant changed, the LLM is not called, the output is kept, and approval remains valid.
- **Patch:** otherwise, the LLM receives the current artifact and only the changed inputs, then returns only the changed parts; code applies the patch.
- **Settled decisions:** answers and accepted defaults from earlier steps are passed downstream, so later steps do not ask again.
- **New checks validate old results:** increasing a step's checks_version makes its previous result stale. The rerun changes nothing and calls no LLM unless the result fails the updated check.

If a step is interrupted (for example, by closing the TUI or terminating the process), completed LLM stage results are written immediately to .q3tui/cache/stages/<step>/. The next run reuses them at no extra cost and runs only the remaining work.

5.2. Gates, Questions, and Decisions

Every step that can ask questions writes conventional default values into its output (marked Assumption:) and lists the questions with default_assumption. Each question has 2–4

suggested options, and each option states its design impact (area / speed / power / complexity). The gate handles them according to its mode (Figure 5-2).

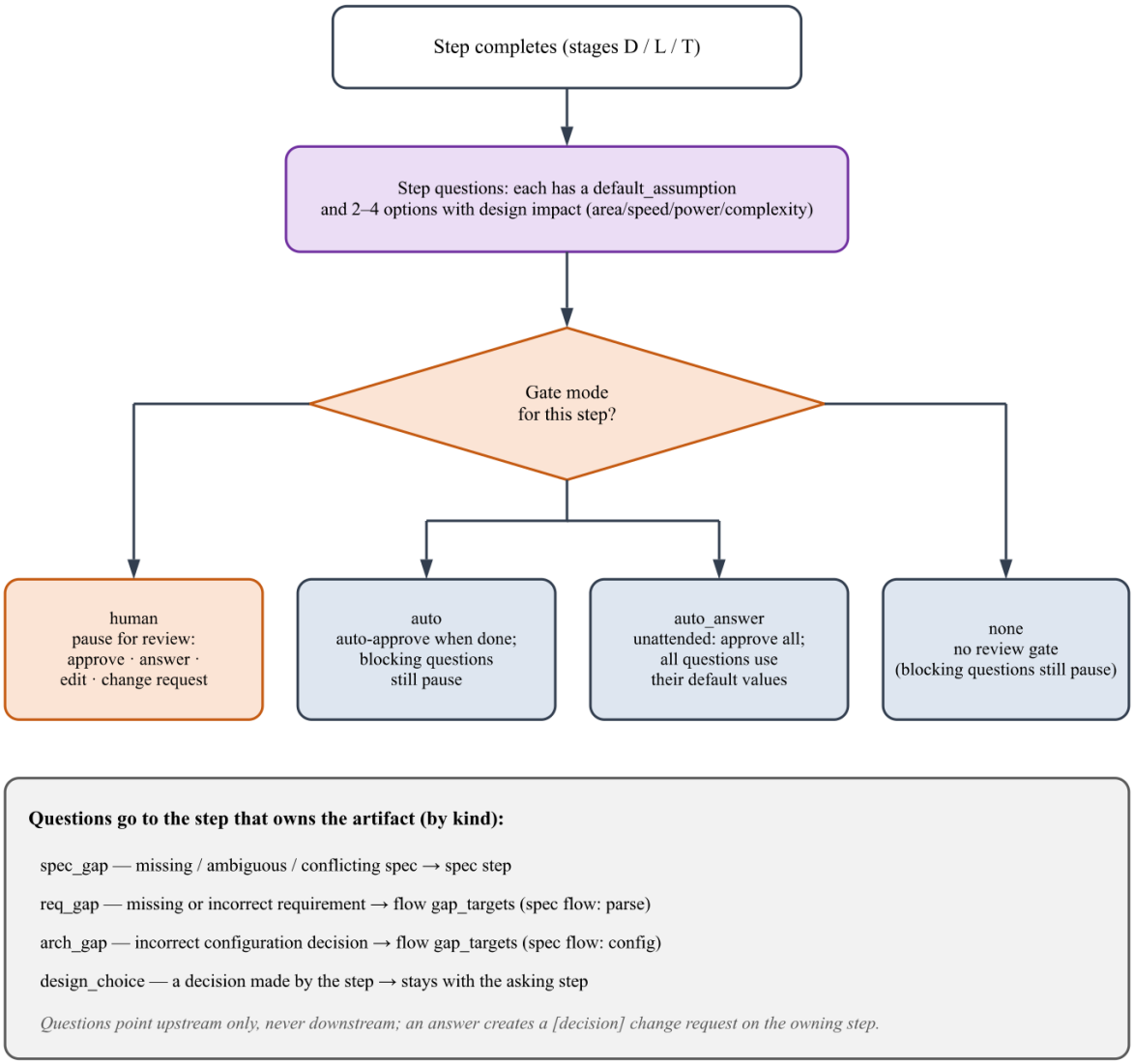


Figure 5-2: Gates, Gate Modes, and Question Routing5-2

Table 5-1: Question Types and Owning Steps5-1

Kind	Meaning	Owning step
`spec_gap`	Spec is missing, ambiguous, or conflicting.	`spec`
`req_gap`	A requirement is missing or incorrect.	Flow gap_targets (spec flow: parse).
`arch_gap`	A structural/configuration decision is wrong or makes the problem infeasible.	Flow gap_targets (spec flow: config).
`design_choice`	A decision made by the asking step.	Remains with that step.

Each artifact has exactly one owning step. Later steps never patch an earlier step's problem or silently change its output. Questions point back to the owning step; answering a gap question creates a [decision] change request on that step, which updates only affected parts

before downstream steps continue. A blocking question has no safe default and cannot be approved by accepting a default.

5.3. Fix Loop

The product does more than report errors: it determines who owns each error and sends a scoped fix request to that step (Figure 5-3).

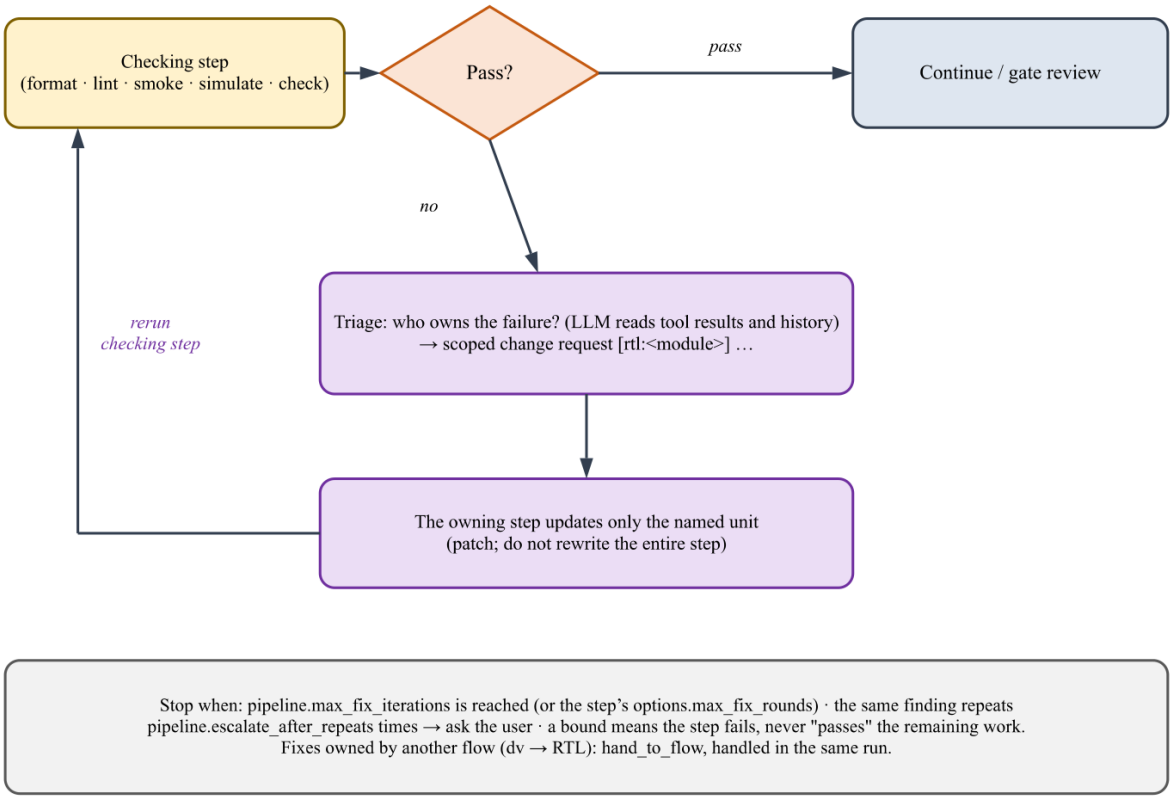


Figure 5-3: Fix Loop Between a Checking Step and Its Owner5-3

After a step sends a fix request upstream, the run continues automatically: the owning step updates only the named units, then the checking step runs again. This repeats until the check passes, a review gate pauses, a blocking question awaits an answer, pipeline.max_fix_iterations is reached, or the same finding returns pipeline.escalate_after_repeats times and becomes a question for the user. Scoped change requests such as [step:unit] … rerun only the named unit (module or test case).

5.4. Deterministic First

Anything computable is handled by code: parsing, rule checks, tool runs, report generation, and tracing. The LLM writes, reasons, and fixes issues, and always receives tool results as

facts (through prompts or in-process MCP tools mcp__q3tui__*). Agents return content as data where possible; code writes the files. Prompts include only what a unit needs, with static content first; fixes use small sessions containing only the problem and current content. Tool output is centrally limited by llm.tool_output_limit.

5.5. RTL and Testbench Independence

This is a product decision: verification is meaningful only when the two sides cannot see each other (Figure 5-4). Hooks enforce the boundary through deny_dirs, write_dirs, and write_files.

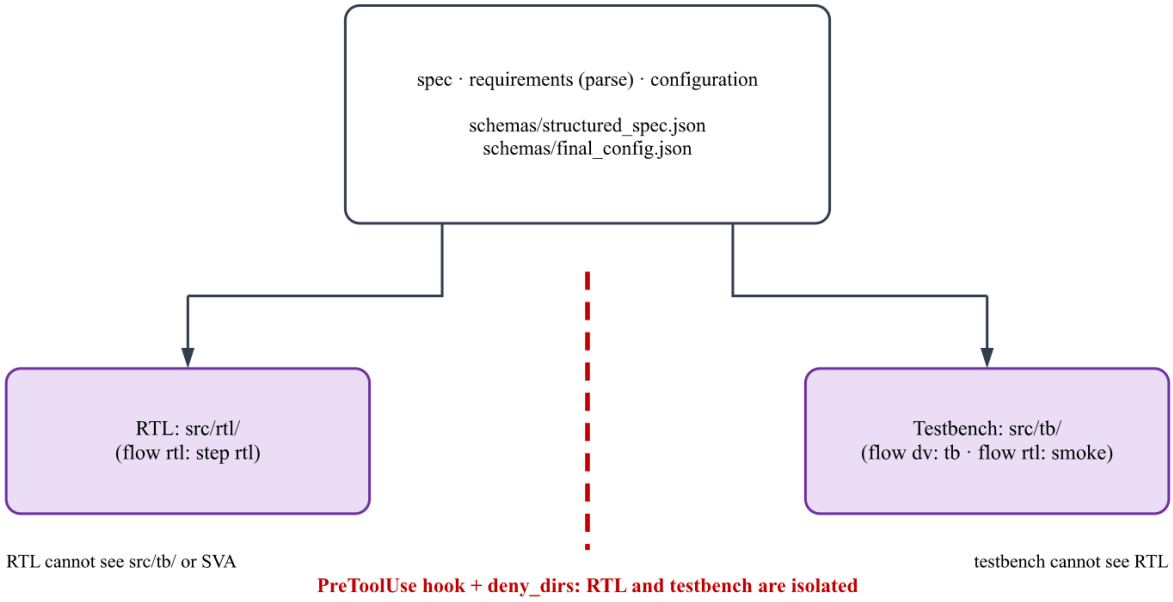


Figure 5-4: RTL and Testbench Independence5-4

Table 5-2: What Each Step Can See (Spec and RTL Flows)5-2

Actor	Can see	Never sees
spec	Intent, spec template, reference documents in spec/.	—
parse, config	Spec (and parse output).	—
rtl	Spec, parse, and config.	`build/ (old tool output), model directory.`
smoke (RTL flow)	Spec, parse, and config (used to write the testbench).	RTL and style-lint results (only code loads RTL for compilation).
`tb (DV flow)`	Spec, requirements, and configuration.	RTL until the final integration stage of the UVM backend.

.q3tui/ is blocked for every stage. For Glob, Q3TUI substitutes mcp__q3tui__glob (safe_glob) so old output cannot be exposed through file listings.

5.6. Team Mode

When team.enabled is on (with fresh sessions), all LLM tasks for a role use one persistent Claude session for that role across tasks, runs, and restarts. The session reads the project once, then reuses its prompt cache at about one-tenth the cost; it remembers why it made earlier choices and can fix bugs in one pass.

Table 5-3: Team Mode Roles (llm/roles.py)5-3

Role	Tasks (stage names)	Cannot see
architect	`spec_* (write and update the spec)`	model, rtl, tb
designer	`rtl_* (RTL modules and fixes)`	model, tb
reviewer	`rtl_review_*; never writes RTL`	model, tb
verifier	`tb_*, vplan* (test plan and testbench)`	rtl, model
flow	Every task in session: flow mode	— (each task keeps its own limits)

Other reviewers, delegated sub-agents, and stages not assigned a role (such as parse_spec) always run fresh. A role's system prompt and tool list stay stable to benefit from prompt caching. Each role has a NOTES.md file for decisions and open issues and compacts its session when the context fills. Hooks enforce isolation for every task.

5.7. Project History

Q3TUI saves a project version whenever a changed step finishes (done or failed), and marks passed/<flow> whenever a flow passes. History is in .q3tui/history/, a separate Git repository that is never pushed and does not touch your .git directory. Steps read it through code: DV triage can see which versions changed RTL since a test last passed. Disable it with pipeline.history: false.

6. Project Directory Structure

Q3TUI works in a project directory, with one directory per design block. Each directory has an owner and role: yours (you edit it; Q3TUI does not delete it), generated (a step writes it; reset deletes it), or disposable (tool output that can be deleted at any time). Steps do not hardcode directory names: they query Project (spec_dir, sdc_dir, fpga_dir, and others), using names from project.* settings.

```
<project>/
├── q3tui.yaml          project configuration (overrides ~/.q3tui/q3tui.yaml)
├── config/             tools.json · pdks.json · rtl_rules.md · skills/
├── flows/  prompts/    project-specific flows and prompts (optional)
├── spec/               intent.md · template.yaml · spec.md · ref/
├── constraints/sdc/    sdc.json and rendered Tcl files
├── src/                rtl/ · tb/ · fpga/ · sw/
├── schemas/            structured_spec.json · final_config.json ·
│                       questions/<step>.* · FPGA/ASIC/CDC/RDC results
├── reviews/            your review decisions (kept on reset)
├── reports/            code-generated reports (sanity, dv_*)
├── docs/               final specification (no flow writes here yet)
├── build/              syn/{fpga,asic} · sim · sanity (tool output)
└── .q3tui/             pipeline.json · cache/ · history/ · runs/ ·
                        team/ · bkp/ · run.<flow>.lock · stats.jsonl
```

Table 6-1: Main Directories and Owners6-1

Directory	Contents	Owner
config/	tools.json (role tools), pdks.json (standard-cell libraries), rtl_rules.md, skills/<name>/SKILL.md	you
spec/	intent.md, spec.md (from the spec step or supplied by you), ref/ (imported documents, read only by spec step)	you
src/rtl/	RTL and filelist.f; csr/, style_lint/, smoke/ are work folders for the RTL flow	generated
src/tb/	DV flow testbench (project.tb_dir)	generated
schemas/	JSON artifacts, questions and answers for all steps, and NOTES.md for flow sessions	generated
reviews/	parse_reviews.json, config_reviews.json, <role>_answers.json; kept on reset	you
build/	Tool output; the only location from which tool output is read	disposable
.q3tui/	Internal state; never visible to LLM stages; can be deleted but history is lost	Q3TUI

Every file you edit manually makes downstream steps stale. Before an agent edits or resets a file, Q3TUI backs it up to .q3tui/bkp/<timestamp>/. Directories imported with q3tui import are blocked for every stage.

7. Configuration and Usage

7.1. Installation

You need uv (the install script can install it; uv downloads Python ≥ 3.12 if the system version is older) and access to an LLM (Claude Code login by default, or another provider described in Section 2.4).

```
curl -fsSL \

https://raw.githubusercontent.com/nguyenquanicd/VLSIT_Q3TUI_ASIC_AI_FLOW/main/i
nstall.sh | bash
q3tui --version
```

This command installs q3tui in ~/.local/bin; run it again to update. Other options include pinning a version with Q3TUI_REF, installing from a clone (./install.sh), previewing or uninstalling (--dry-run, --uninstall), or using uv tool install git+https://github.com/nguyenquanicd/VLSIT_Q3TUI_ASIC_AI_FLOW.git. EDA tools are provided by the user: q3tui tools init writes a reference config to config/tools.json, and q3tui tools check shows which roles the current flow needs and which are missing.

7.2. Configuration Layers

Table 7-1: Configuration Precedence (Later Layers Win)7-1

#	Layer	Notes
1	Built-in defaults	
2	`$Q3TUI_HOME/q3tui.yaml (defaults to ~/.q3tui)`	Site settings: tool modules and default model.
3	`<project>/q3tui.yaml`	Per-design overrides.
4	`q3tui --config <path> or $Q3TUI_CONFIG`	
5	Command-line flags: --model, --effort, …	

Configuration layers are deep-merged; unknown keys are rejected. In the TUI, press , (or use /settings) to open Settings, with General, Gates, LLM, Spec, RTL, and per-step Models tabs. Changes are schema-checked, take effect from the next stage, and are written to <project>/q3tui.yaml. q3tui config show prints the merged configuration and the source of each value.

7.3. Quick Start

```
mkdir my_block && cd my_block
q3tui init            # writes spec/intent.md: describe the block you want (or
place your specification in spec/)
q3tui                 # open the TUI; press r to run
q3tui --flow rtl run  # in the same project, now generate and check RTL
q3tui status          # steps, gates, and open questions
q3tui auto            # or run the entire flow chain unattended
```

In the TUI, r runs the flow, a opens review (approve, answer questions, edit, or request a change), o shows open questions, and ? lists all keys. The examples/ directory contains sample projects (apb_slave, apb_dual_clk, axi4_sram_ctrl): copy one, cd into it, and run q3tui.

7.4. Main CLI Commands

Table 7-2: Main q3tui Commands7-2

Command	Purpose
`q3tui init, q3tui`	Initialize a project; open the TUI.
`q3tui run [--intent …] [--spec …] [--import …] [--from/--to/--only] [--regenerate STEP] [--yes]`	Run missing or stale steps.
`q3tui status, questions, answer ID "…", approve, change`	View status; handle questions, gates, and change requests.
`q3tui gate STEP MODE, reset, cost, stats`	Change gate mode; reset a step; view cost and statistics.
`q3tui auto, q3tui --flow sanity loop`	Run the full flow chain; run the sanity fix loop.
`q3tui import DIR`	Import an existing project directory (spec and RTL; tests/testbench are categorized but not yet consumed by DV tb).
`q3tui history log|diff|show|restore`	Project version history.
`q3tui tools init|check, q3tui flow list|check`	Role tools; validate flow files.
`q3tui gateway status|start|stop|models|install| login, q3tui keys list|set|clear`	LiteLLM gateway and API keys.
`q3tui config show, q3tui team status|reset, q3tui design parse|check, q3tui act NAME key=value`	Merged configuration; team mode; parse/check design; invoke an Ops action.

7.5. Development

Run uv sync and then uv run pytest. Tests do not call a live LLM; they replace q3tui.llm.runtime.run_stage with tests/fakes.FakeLLM. EDA tools are usually not needed (tests check command construction and log parsing `without running tools). Add new`

user actions to core/ops.py and expose them through both TUI keys and assistant tools.

8. Current Limitations and Roadmap

Q3TUI is in Beta (0.1.0). Table 8-1 lists known limitations and places where repository documentation does not match the source code.

Table 8-1: Current Limitations8-1

Area	Description
Windows	Q3TUI supports Linux and Windows through core/compat.py, but Windows still has path-handling issues in shlex-built commands and YAML, so it does not yet run end to end (a development finding, not a repository specification).
EDA tools	VCS, Design Compiler, Formality, VC Static, Vivado, and others are Linux-only; Q3TUI does not install them, and an unconfigured role is reported as N/A.
UVM	The DV UVM backend currently covers only AXI-style memory-mapped IP with a VIP; other IP is rejected with the relevant parameters.
Import	Testbench/test files are recognized but not yet consumed by the DV tb step (reported as "not imported"); SVA, scripts, and filelists are skipped.
Sanity	GCA is not available (no tool); only CDC and RDC via VC Static are supported.
Future directions	According to docs/spec/README.md: formal verification, upgrading existing RTL, and a web UI.
Providers other than Claude	Claude Code prices only models it recognizes, so a stage using another provider's model may report $0; thinking_budget is available only for Claude models.
Documentation mismatches	`CLAUDE.md still mentions lanes (core/lanes.py, q3tui tag, docs/spec/lanes.md), but those files are absent and core/config.py says lanes were removed (replaced by pipeline.design_flows and concurrent flows). docs/spec/architecture.md says UVM is out of scope, although the DV flow already has a UVM backend.`

Appendix A. Acronyms and Terms

Table 8-2: Acronyms and Terms8-2

Term	Meaning
CDC / RDC	Clock Domain Crossing / Reset Domain Crossing
CSR	Control and Status Register
DUT	Design Under Test
Gate	A step pause where the user reviews and makes a decision
LEC	Logic Equivalence Check (Formality)
PPA	Power, Performance, Area
REQ-NNN	Requirement identifier
SDC	Synopsys Design Constraints
Step kind	Registered step type (steps.KINDS); flows select kinds to create steps
Role (tool)	EDA capability (lint, synth, sim, etc.) called by a step; tools.json maps it to a specific command
TUI	Terminal User Interface (Textual)
UVM / VIP	Universal Verification Methodology / Verification IP